

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1153

COURSE OUTCOME COVERED

CO4:

implement object-oriented system layers using design principles and patterns, and integrate access and view layers with OODBMS (BL5)

Assessment Tool Weightage: 40%

QUESTIONS: 1

Total Marks:10

Annexure A

Industry Problem Solving Task

Code of Conduct:

I ___D RAGHU VARDHAN 192311051_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

D RAGHU VARDHAN

Industry Problem Solving Task

1. A mid-sized **e-commerce company** is building a scalable shopping platform to handle thousands of users daily. The system follows a **layered object-oriented architecture**, where the **presentation layer** provides web and mobile interfaces, the **business logic layer** processes orders, payments, and inventory, and the **access layer** communicates with an Object-Oriented Database Management System (OODBMS) to store product and customer data.

To enhance flexibility, the system uses **MVC pattern** for UI separation, **Singleton pattern** for managing database connections, and **DAO pattern** for data access. The design ensures that new features like discount modules can be added without modifying existing code. Performance is improved using caching and efficient query handling.

Question:

Analyze how design principles, layered architecture, and design patterns are applied in this system to ensure scalability and maintainability, and explain how the layers are integrated with OODBMS.

Analysis of Design Principles, Layered Architecture, and Design Patterns in a Scalable E-Commerce Platform

Introduction

A mid-sized e-commerce company requires a scalable and maintainable shopping platform capable of handling thousands of users every day. To achieve this goal, the system is designed using Object-Oriented System Development principles, a layered architecture, and several design patterns. The architecture separates responsibilities into different layers, allowing each layer to focus on a specific task. The system uses an Object-Oriented Database Management System (OODBMS) for storing and managing product, customer, and order objects efficiently. Design patterns such as MVC, Singleton, and DAO improve flexibility, reusability, and maintainability.

1. Application of Object-Oriented Design Principles

Encapsulation

Encapsulation is used to protect data and ensure controlled access to object properties. Classes such as Product, Customer, Order, and Payment contain both data and methods required to manipulate that data.

Example:

- Product object contains product details and inventory methods.
- Customer object contains customer information and account operations.
- Order object contains order details and processing methods.

Benefits:

- Improves security.
 - Prevents unauthorized access.
 - Makes the system easier to maintain.
-

Abstraction

Abstraction hides complex implementation details from users and developers working on other modules.

Example:

A customer places an order through the website without knowing how inventory checking, payment validation, and database storage are performed internally.

Benefits:

- Reduces system complexity.
 - Simplifies development.
 - Improves usability.
-

Inheritance

Inheritance allows new classes to reuse existing functionality.

Example:

- User class acts as a parent class.
- Customer and Admin classes inherit from User.

Benefits:

- Promotes code reuse.
 - Reduces redundancy.
 - Simplifies maintenance.
-

Polymorphism

Polymorphism allows the same operation to behave differently depending on the object type.

Example:

Different payment methods such as Credit Card, Debit Card, and UPI implement a common Payment interface.

Benefits:

- Increases flexibility.
 - Supports future payment methods easily.
-

Open-Closed Principle

The system is designed to be open for extension but closed for modification.

Example:

A Discount Module can be added without changing existing order processing code.

Benefits:

- Easier feature enhancement.
 - Reduced risk of introducing errors.
-

2. Layered Architecture

The application follows a three-layer architecture that separates responsibilities and improves scalability.

A. Presentation Layer

The Presentation Layer provides user interfaces through web browsers and mobile applications.

Responsibilities:

- Display products and categories.
- Accept user inputs.
- Show shopping cart and order details.
- Display payment confirmations.

Components:

- Web Interface
- Mobile Application
- Controllers

Advantages:

- Better user experience.
 - Independent UI development.
 - Easy modification of interface designs.
-

B. Business Logic Layer

The Business Logic Layer contains the core functionality of the application.

Responsibilities:

- Order processing.
- Payment validation.
- Inventory management.
- Discount calculation.
- Customer authentication.

Example Flow:

1. Customer places an order.
2. System validates stock availability.
3. Payment is processed.
4. Inventory is updated.
5. Order confirmation is generated.

Advantages:

- Centralized business rules.
 - Easy maintenance.
 - Improved scalability.
-

C. Access Layer (Data Layer)

The Access Layer manages communication with the OODBMS.

Responsibilities:

- Store product information.
- Retrieve customer records.
- Update inventory.
- Manage order data.

Advantages:

- Isolates database operations.
 - Simplifies database management.
 - Supports future database changes.
-

3. Design Patterns Used

MVC (Model-View-Controller) Pattern

The MVC pattern separates application components into three parts.

Model

Represents business objects and data.

Examples:

- Product
- Customer
- Order
- Payment

View

Displays information to users.

Examples:

- Product pages
- Shopping cart screens
- Mobile app interfaces

Controller

Handles user requests and coordinates actions between View and Model.

Examples:

- Login Controller
- Product Controller
- Order Controller

Benefits:

- Clear separation of concerns.
 - Easier maintenance.
 - Improved code organization.
 - Supports multiple user interfaces.
-

Singleton Pattern

The Singleton pattern ensures only one instance of a database connection manager exists.

Example:

DatabaseConnectionManager

Benefits:

- Reduces memory consumption.
 - Prevents multiple unnecessary connections.
 - Improves performance.
 - Centralized connection management.
-

DAO (Data Access Object) Pattern

DAO provides an abstraction layer between business logic and database operations.

Example DAOs:

- ProductDAO
- CustomerDAO
- OrderDAO

Common Methods:

- save()
- update()
- delete()
- find()

Benefits:

- Database-independent business logic.
 - Easier testing.
 - Improved maintainability.
 - Better flexibility.
-

4. Integration of Layers with OODBMS

The OODBMS stores objects directly instead of converting them into relational tables.

Working Process**Step 1**

Customer browses products using the web or mobile application.

Step 2

The Presentation Layer sends requests to the Business Logic Layer.

Step 3

Business Logic processes the request and applies business rules.

Step 4

DAO objects communicate with the OODBMS.

Step 5

Product, Customer, and Order objects are retrieved or stored.

Step 6

Results are returned to the Business Layer and displayed to the customer.

OODBMS Advantages

- Direct storage of objects.
 - Supports object relationships naturally.
 - Handles inheritance efficiently.
 - Eliminates complex object-relational mapping.
 - Faster object retrieval.
-

5. Scalability and Performance Enhancement

Caching Mechanism

Frequently accessed products and category information are stored in cache memory.

Benefits:

- Faster response time.
 - Reduced database load.
 - Improved user experience.
-

Efficient Query Handling

Optimized object retrieval techniques are used to reduce processing time.

Benefits:

- Faster search operations.
 - Improved transaction processing.
 - Better system performance under heavy load.
-

Modular Expansion

The architecture supports easy addition of new modules.

Examples:

- Discount Module
- Coupon Management
- Loyalty Program
- Recommendation System

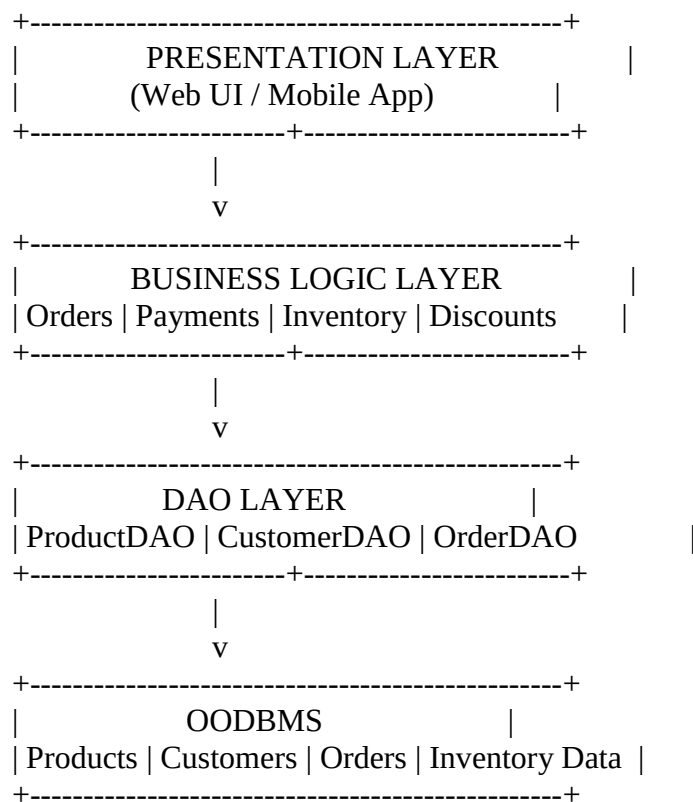
Benefits:

- Future-ready design.
- Minimal code modification.
- Reduced development effort.

Conclusion

The e-commerce platform successfully applies **object-oriented design principles**, **layered architecture**, and **design patterns** to achieve scalability, flexibility, and maintainability. The **Presentation Layer** handles user interaction, the **Business Logic Layer** processes core operations, and the **Access Layer** communicates with the **OODBMS**. Patterns such as **MVC**, **Singleton**, and **DAO** ensure clear separation of concerns and efficient resource management. Furthermore, caching, optimized queries, and modular design improve performance and allow the platform to handle thousands of users while supporting future enhancements with minimal changes.

E-COMMERCE PLATFORM



RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding ; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation